

Assignment 1: Promoter Discovery in Bacteria

Salmonella enterica subsp. *enterica* strain PartC-Senterica-RM8376 (accession ASM2286996v1)

Index No. 220472T • 2 October 2026

1 Introduction

A promoter is the short region of DNA in front of a gene where RNA polymerase binds and starts to make mRNA. In bacteria most genes are read by RNA polymerase together with the sigma factor σ^{70} , which recognises two short boxes: the -35 box (consensus TTGACA) and the -10 box or Pribnow box (consensus TATAAT) [1, 2]. The boxes sit about 35 and 10 bases before the transcription start site (TSS) and are about 17 bases apart. The -10 box is where the two DNA strands are pulled apart, so it is rich in A and T (an A–T pair is held by two hydrogen bonds while a G–C pair has three). In the IUPAC code the letter W (“weak”) means A or T, so TATAAT can be written as WWWWWW. In this assignment the promoter is modelled in an even simpler way, as the query WWWWW.

From a signal processing point of view a genome is a very long discrete signal that can take only four values (A, C, G and T), and finding a promoter is like detecting a short known pattern inside a long noisy signal [3]. I used the two detectors from the module: a standard local search (exact pattern matching with the Smith–Waterman algorithm) and a statistical alignment with a position probability matrix (PPM), which works like a matched filter. My aims were to

1. cut 100 bases upstream and 3 bases downstream of 1000 sense-strand genes and check the extraction using the start codon (Methionine),
2. find WWWWW with the local search (Q1) and count the consecutive Ws (Q2),
3. build a PPM from the hits (Q3) and use it for a statistical alignment (Q4), and
4. compare the two searches (Q5).

Figure 1 shows the part of the DNA that I worked with for every gene.

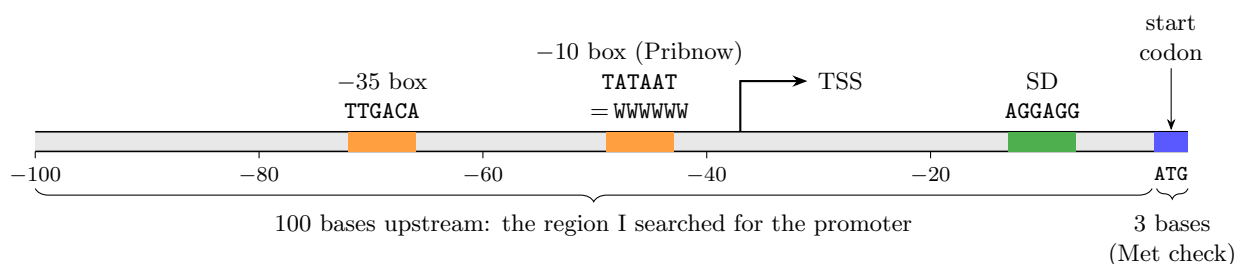


Figure 1: The 103-base sequence that I cut out for every gene. The two promoter boxes are about 17 bases apart. The positions of the -35 and -10 boxes, the transcription start site (TSS) and the Shine–Dalgarno ribosome binding site (SD) are only typical; they change from gene to gene, and some genes have no promoter of their own.

2 Data used

I downloaded the genome package of my accession (ASM2286996v1) from NCBI Datasets [4]. The package has many files, but I only needed the two shown in Table 1. All the analysis was done in Python 3 (NumPy, pandas, Matplotlib and SciPy). I split the code into one file per step of the

assignment; `main.py` runs everything in one go, and a Jupyter notebook runs the same functions step by step. Both are in the submitted `.zip` file (Appendix A).

Table 1: The data files I used and why.

File	What it contains	Why I used it
GCA_022869965.1_ASM2286996v1_genomic.fna (GenBank genome, FASTA)	The DNA of the chromosome (4,857,492 bases) and of a 93,933-base plasmid, written as A, C, G and T.	This is the “signal” that I searched. All 1000 sequences were cut out of it.
ncbi_dataset.tsv (protein table)	One row per gene: chromosome, start (Begin), end (End), strand (Orientation), gene type, protein length, locus tag and name.	The genome itself has no labels, so this table tells me where each gene starts and on which strand. I also used the protein length to check my code.

The other files in the package were not needed. The GFF, GTF and GenBank flat files hold the same gene annotation as the table in more complicated formats, and the CDS and protein FASTA files only contain the genes themselves. The CDS file in particular cannot be used for this assignment because it does not include the DNA in front of each gene, which is exactly where a promoter is.

One detail confused me at first: the table calls the chromosome `NZ_CP064385.1` (its RefSeq name) while the GenBank FASTA calls it `CP064385.1`. A RefSeq record is a copy of the GenBank record, and I checked that the RefSeq and GenBank FASTA files in the package have exactly the same sequence (same length and same MD5 checksum), so the positions in the table are valid for the GenBank genome. The chromosome has 52.2% G+C. The table lists 4,850 genes (4,612 protein-coding, 113 pseudogenes and 125 RNA genes); 4,733 of them are on the chromosome, 2,280 on the + strand and 2,453 on the – strand. I only used the chromosome.

3 Method and why I chose it

3.1 Choosing the genes and cutting the sequences

The brief asks for genes of the sense strand, which I took to mean the genes on the + strand of the chromosome (`Orientation = plus`). For these genes the DNA is read from left to right, so “upstream” simply means smaller positions and nothing has to be reverse-complemented. I kept only protein-coding genes, because pseudogenes and RNA genes do not make a protein and therefore cannot be checked with Methionine. One gene crosses the end of the circular chromosome (its `Begin` is larger than its `End`), so I skipped it. From the remaining 2,153 genes I took the first 1000 in the order they appear on the chromosome, from *murJ* (starting at base 571) to *acpT* (ending at base 2,496,884). I did not choose genes by hand, so the selection is not biased towards genes with or without promoters.

The table uses 1-based positions (the first base of the genome is 1) but Python strings start at index 0. For a gene whose first base is b , the start codon is at positions b to $b + 2$, so in Python the 103-base sequence is `genome[b-101 : b+2]`, which covers positions $b - 100$ to $b + 2$: 100 upstream bases followed by the start codon. I used 100 upstream bases because in *E. coli*, a close relative of *Salmonella*, the TSS is usually 20–40 bases in front of the start codon [5], so the –10 and –35 boxes should normally fall inside this window (Figure 1). In this report I give positions relative to the start codon: the base just before it is –1 and the farthest base is –100. The position of a hit is the position of its first (5′) base, so a WWW hit at –90 covers –90 to –87.

3.2 Checking the extraction with Methionine

An off-by-one mistake in the indexing would shift every sequence, so before anything else I checked the 3 downstream bases. If the positions are right they must be a start codon, which codes for Methionine. Table 2 shows nine well-known genes: all of them have ATG, and their translated proteins start with M and have exactly the length given in the table. Over all 1000 genes the start codons were ATG (912), GTG (66), TTG (18), ATT (2) and CTG (2). The non-ATG codons are normal in bacteria: at the start of a gene they are read by the initiator tRNA that carries formyl-Methionine, so every protein still starts with Methionine. As a stronger test I translated every whole coding sequence. 999 of the 1000 gave exactly the protein length in the table and ended with a stop codon. The only exception was an IS3-family transposase (IUJ38_RS08235), which is known to need a ribosomal frameshift. This convinced me that my indexing is correct.

Table 2: Methionine check on known genes. Upstream bases in lower case, start codon in capitals.

Gene	Locus tag	Last 12 bases + start codon	Start of my translation	Length (aa): table / mine
<i>murJ</i>	IUJ38_RS00010	aatacaaaagaa ATG	MNLLKSLAAV...	511 / 511
<i>rpoD</i>	IUJ38_RS10490	ggataccgtctt ATG	MEQNPQSQLK...	615 / 615
<i>crp</i>	IUJ38_RS11775	ggataaccgcg ATG	MVLGKPQTDP...	210 / 210
<i>hns</i>	IUJ38_RS02980	gagattactaca ATG	MSEALKILNN...	137 / 137
<i>fis</i>	IUJ38_RS11365	gctgacagaact ATG	MFEQRVNSDV...	98 / 98
<i>ompC</i>	IUJ38_RS01830	aaggcatataaa ATG	MKKVVVLSAV...	372 / 372
<i>hilA</i>	IUJ38_RS08775	tacactattatc ATG	MPHFNPVPVS...	553 / 553
<i>tolC</i>	IUJ38_RS10355	tcacaacaagga ATG	MQMKLLPIL...	491 / 491
<i>trpA</i>	IUJ38_RS02860	aggggaaatctg ATG	MERYENLFAQ...	268 / 268

3.3 Q1: standard local search

Because I am looking for a short query somewhere inside a longer sequence, I used local alignment (the Smith–Waterman algorithm [6]) and not global alignment, which would try to align the two sequences from end to end. For the upstream sequence x (100 bases) and the query $q = \text{WWW}$, the algorithm fills a score matrix

$$H(i, j) = \max\{0, H(i-1, j-1) + s(x_i, q_j), H(i-1, j) + g, H(i, j-1) + g\}, \quad (1)$$

where $s = +1$ when the base fits the query letter (A or T for W) and $s = -1$ otherwise, and the gap penalty is $g = -2$. These are the same scores as in the local search code given in the module. The 0 in Equation (1) lets an alignment start anywhere, which is what makes the search local, and the best alignment is found by tracing back from the cell with the highest score.

The query is *intact* when all of WWW is aligned with no mismatch and no gap. With these scores the highest possible score is 4 and only an intact match can reach it, so I counted a gene as having a potential promoter when its best score was 4. When a sequence has several intact matches they all score 4; like `max()` in MATLAB or NumPy, my code takes the first maximum, which is the hit nearest the 5' end (farthest from the gene). I searched only the 100 upstream bases, because a promoter cannot overlap the start codon.

3.4 Q2: consecutive Ws

For every hit I counted how many A or T bases follow each other from the first base of the hit towards the gene. A Pribnow box (TATAAT) would give 6, so this tells me whether the WWW is part of a longer A/T stretch. To judge the result I compared it with random DNA. If every base is W with probability p , independently of the others, a run that already has 4 Ws continues with probability p at every step, so its length L follows a geometric distribution

$$P(L = n) = p^{n-4}(1 - p), \quad n = 4, 5, 6, \dots \quad (2)$$

I used $p = 0.549$, the A/T fraction of my 1000 upstream regions (the whole genome has only 0.478).

3.5 Q3: position probability matrix

From each hit I took the 6 bases starting at the hit (whatever their C/G content, as the brief says), because the Pribnow box is 6 bases long. I counted how many times each base b appears at each position j (the position frequency matrix $n_{b,j}$) and turned the counts into probabilities

$$p_{b,j} = \frac{n_{b,j} + k}{N + 4k}, \quad k = 0.01, \quad (3)$$

where $N = 995$ is the number of 6-base sequences. The small pseudocount k is needed because some bases never appear at some positions and $\ln 0$ would break the scores in Q4; 0.01 is the value used in the module code. To see how much each position tells us, I calculated its information content against a random base (probability 1/4 each),

$$I_j = \sum_{b \in \{A,C,G,T\}} p_{b,j} \log_2 \frac{p_{b,j}}{0.25} \text{ bits}, \quad (4)$$

which is 0 for a completely random position and 2 bits for a position that is always the same base, and I drew a sequence logo [7].

3.6 Q4: statistical alignment

In the statistical alignment I slid a 6-base window along each upstream sequence (95 windows, from -100 to -6) and scored every window with the log-likelihood of its bases under the PPM:

$$S(m) = \sum_{j=1}^6 \ln p_{x_{m+j-1}, j}. \quad (5)$$

This is like a matched filter: the PPM is the template, sliding it along the sequence is a correlation, and the window with the largest output is the best alignment [8]. The highest score any window can get is the score of the consensus (the most likely base at every position), $S_{\text{cons}} = \sum_j \ln \max_b p_{b,j}$. I said that a promoter is present when $S_{\text{best}} - S_{\text{cons}} \geq T$ and tried $T = -1, -2, -3, -4$ and -5 , the same range used in the module. For example, $T = -2$ means that the best window is at least $e^{-2} \approx 14\%$ as likely as the consensus. For the comparison of positions in Q5 I used $T = -2$.

3.7 Chance-level control

A detector is only useful if it finds the pattern more often in real promoter regions than by chance. To measure the chance level, I shuffled every upstream sequence 100 times (same bases, random order; 100,000 sequences in total) and ran both searches on the shuffled sequences. Shuffling keeps the A/T content but destroys any real signal, so the shuffled result is what each detector reports when there is nothing to find. For the shuffled sequences I used a regular expression instead of Smith–Waterman to save time; on the real sequences the two gave exactly the same hits. To compare the two position distributions in Q5 I used the two-sample Kolmogorov–Smirnov (KS) test.

4 Results and discussion

4.1 Q1: local search for WWW

An intact WWW was found in 995 of the 1000 upstream sequences, so **99.5%** of the genes have a potential promoter by this definition. The other five genes (*holB*, IUJ38_RS03200, *pduS*, *srlE* and *gltB*)

only reached a score of 3. Their upstream regions are G+C rich (61–66% G+C), and for the first four the whole region lies inside the coding sequence of the gene before them, which ends right on top of their start codon.

Figure 2 shows where the hits are. The distribution is strongly skewed to the far end of the region: the median position is -90 (interquartile range -97 to -77), 135 hits are exactly at -100 and 72.4% of the hits are at -80 or further upstream.

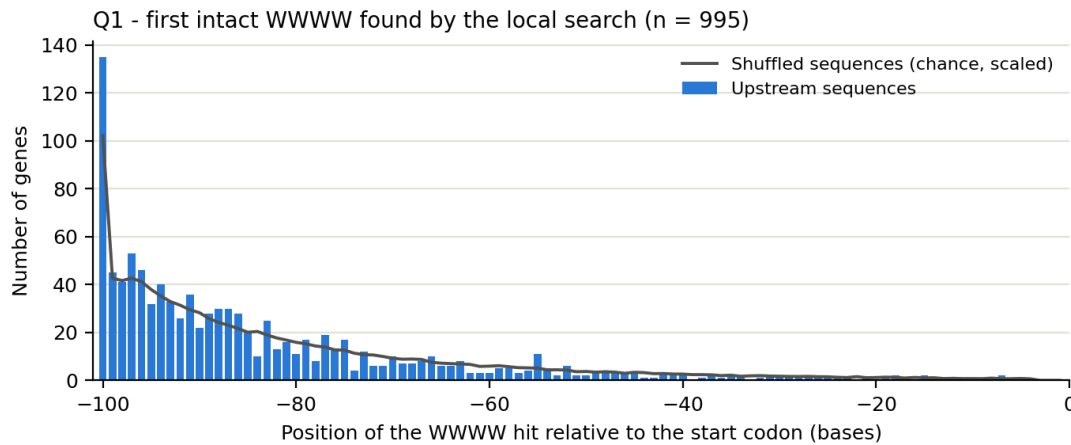


Figure 2: Q1: position of the first intact WWWW found by the local search (bars). The line is the same distribution for the shuffled sequences, scaled to 995 genes.

At first this looked as if the promoters were far away from the genes, but it is really caused by the way the search works. Each upstream sequence contains on average 12.2 intact WWWW (median 11, range 0–42) and the local search reports only the first one it meets, starting from -100 . Figure 2 is therefore just the “waiting time” until the first A/T run, and the shuffled sequences give almost the same curve (median -88). Figure 3 shows all 12,224 occurrences instead of only the first one. They are spread evenly, at about 12% per position, up to -26 , rise to 15–17.5% between -25 and -15 , drop to 4–8% for words starting between -13 and -9 , and rise again to about 16% just before the start codon. The dip is the G-rich Shine–Dalgarno (SD) ribosome binding site (consensus AGGAGG) [9], which also shows up clearly as a drop in A/T content in the lower panel. If I had taken the last hit (nearest the gene) instead of the first, the median position would have been -14 , which is also in this translation start region and not at a -10 box.

4.2 Q2: number of consecutive Ws

The A/T runs at the hits are 4 to 22 bases long, with a mean of 5.5 and a median of 5 (Table 3, Figure 4). Most runs are short, as expected for random DNA, but compared with the geometric distribution of Equation (2) there are fewer runs of exactly four (381 against 449 expected) and more long runs (122 runs of 8 or more against 90). The longest runs are 22 bases long, upstream of *chbC* (AATTAATTATTTTAAATTTT) and *ftnA*. The real sequences also contain more WWWW words than their shuffled versions (12.2 against 9.7 per sequence). So the upstream regions do contain real A/T tracts. Such tracts are common in bacterial promoter regions because they make the DNA easier to open, but a long A/T run alone is not enough to call a promoter.

Table 3: Q2: number of consecutive Ws at the 995 hits, observed and expected for random DNA (Equation 2, $p = 0.549$).

Consecutive Ws	4	5	6	7	8	9	10	11	12	13	14	≥ 15
Genes (observed)	381	247	157	88	53	33	16	6	4	6	2	2
Expected (random)	448.6	246.3	135.3	74.3	40.8	22.4	12.3	6.8	3.7	2.0	1.1	1.4

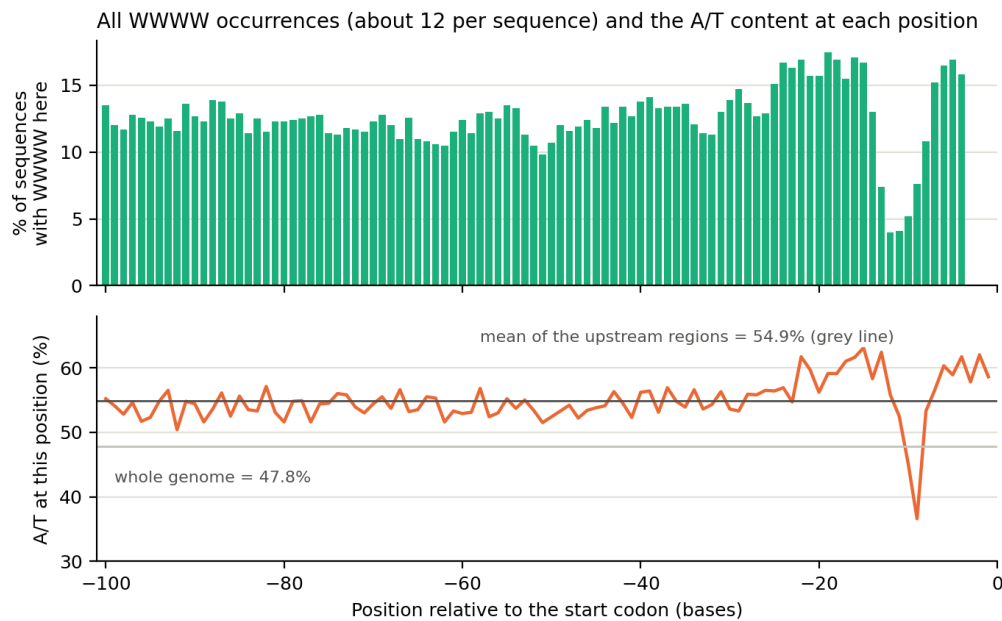


Figure 3: Top: percentage of the 1000 sequences with an intact WWW starting at each position (all occurrences, not only the first). Bottom: A/T content at each position. The dip at -13 to -9 is the Shine–Dalgarno site.

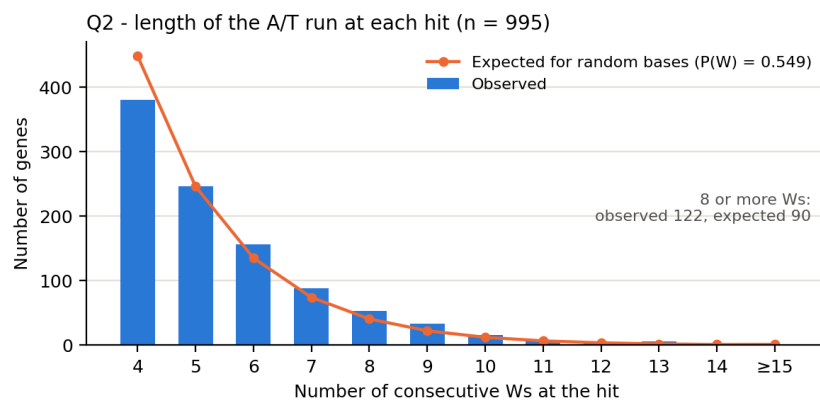


Figure 4: Q2: distribution of the number of consecutive Ws at each hit (bars) and the geometric expectation for random DNA (line).

4.3 Q3: position probability matrix

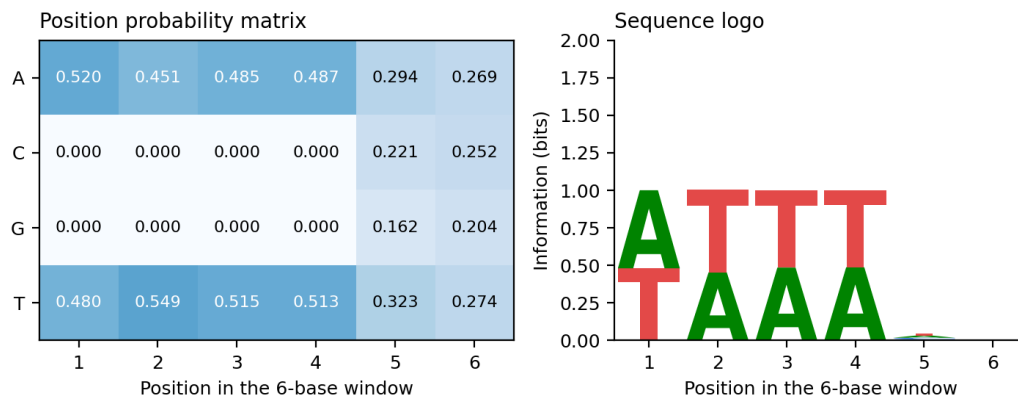
The 995 six-base sequences (239 different ones; the most common were AAAAAA 18, TTTTTT 15 and TTTTTG 15) gave the matrix in Table 4 and the logo in Figure 5. Positions 1–4 contain only A and T, in almost equal amounts, because every sequence starts with the WWW found in Q1. Each of these positions carries about 1 bit, the maximum for a choice between two letters. Positions 5 and 6 are close to random (0.05 and 0.01 bits; A or T 62% and 54% of the time). The consensus is ATTTTT with a score of -5.012 , and the whole matrix holds only 4.06 bits. In other words, the PPM mostly gives back the query that I used to find the hits. I also noticed that the W frequency at position 5 (0.617) is exactly the fraction of hits whose run is longer than four bases ($614/995$), so the PPM and the Q2 distribution describe the same A/T runs.

4.4 Q4: statistical alignment

Table 5 shows the percentage of genes whose best window passed each threshold. With $T = -1$, 99.0% of the genes were detected, and with $T = -2$ to -5 , 99.5%. Every window that passed started with

Table 4: Q3: position probability matrix (counts in brackets) and information content. C and G at positions 1–4 were never seen, so they only have the pseudocount ($p = 1.0 \times 10^{-5}$).

Base	1	2	3	4	5	6
A	0.520 (517)	0.451 (449)	0.485 (483)	0.487 (485)	0.294 (293)	0.269 (268)
C	0.000 (0)	0.000 (0)	0.000 (0)	0.000 (0)	0.221 (220)	0.252 (251)
G	0.000 (0)	0.000 (0)	0.000 (0)	0.000 (0)	0.162 (161)	0.204 (203)
T	0.480 (478)	0.549 (546)	0.515 (512)	0.513 (510)	0.323 (321)	0.274 (273)
Information (bits)	1.00	1.01	1.00	1.00	0.05	0.01

**Figure 5:** Q3: the PPM (left) and the sequence logo (right), where the height of each letter is its probability times the information content of the position.

an intact **WWW**. The reason is the pseudocount: a C or G at positions 1–4 has $p \approx 10^{-5}$, which costs about 10.9 (in ln units) compared with the consensus, much more than any threshold I used. The five genes without **WWW** had best scores 10.9 to 11.2 below the consensus and were not detected. On the other hand, any window that starts with an intact **WWW** scores within 1.37 of the consensus, so every threshold of -1.37 or lower selects exactly the same genes as the local search.

The best windows are spread over the whole region (median -51 , interquartile range -76 to -28 ; Figure 6) and are mostly T-rich words close to the consensus: **ATTTTT** (113), **ATTATT** (64), **ATTTTA** (56) and **TTTTTT** (43). 68.5% of them contain only A and T.

Table 5: Q4: percentage of genes detected by the statistical alignment for each threshold T (best score minus consensus score), for my sequences and for the shuffled sequences.

Threshold T	–1	–2	–3	–4	–5
Upstream sequences	99.0%	99.5%	99.5%	99.5%	99.5%
Shuffled sequences	96.2%	97.6%	97.6%	97.6%	97.6%

4.5 Q5: comparison of the two searches

Table 6 puts the main numbers of the two searches side by side.

Percentage of promoters detected. Both searches find a potential promoter in front of almost every gene (99.5%), and with $T \leq -2$ they find exactly the same 995 genes. This agreement is not a coincidence. The PPM was made only from intact **WWW** hits, so it gives C and G at positions 1–4 a probability of about 10^{-5} , and the statistical alignment can only accept windows that the local search would also accept. It can never be more sensitive than the local search, and with $T = -1$ it is slightly less sensitive (990 genes).

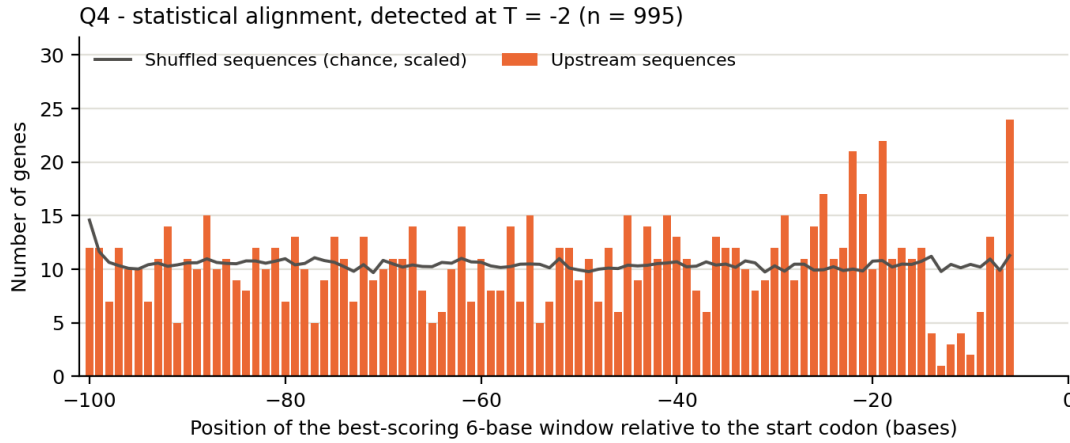


Figure 6: Q4: position of the best-scoring window for the 995 genes detected at $T = -2$ (bars) and the same for the shuffled sequences (line, scaled).

Table 6: Q5: local search (Q1) against statistical alignment (Q4, $T = -2$).

	Local search	Statistical alignment
Genes with a potential promoter	995 (99.5%)	995 (99.5%); 990 at $T = -1$
Same test on shuffled sequences	97.8%	97.6%
Position of the hit: median (IQR)	-90 (-97 to -77)	-51 (-76 to -28)
Mean A/T run length at the hit	5.5	6.4
Same position in both searches	179 genes (18.0%); within 2 bases: 241 (24.2%)	
KS test between the two positions	$D = 0.53, p < 10^{-100}$	

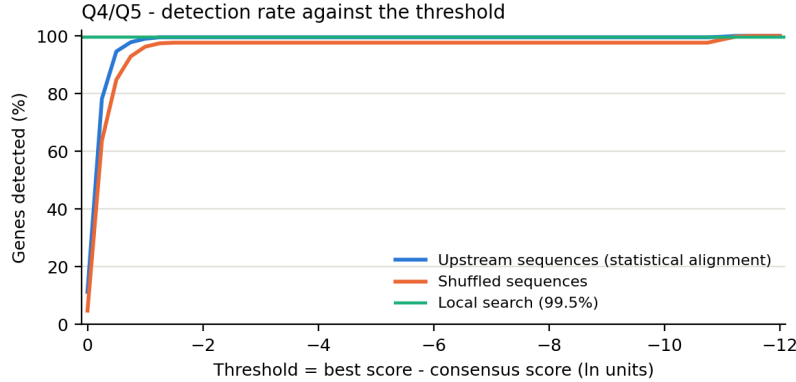


Figure 7: Percentage of genes detected by the statistical alignment as the threshold changes, for my sequences and for the shuffled sequences, compared with the local search.

The more important result is that both percentages are almost the same as the chance level: 97.8% of the shuffled sequences contain **WWW** and 97.6% pass the statistical alignment at $T = -2$. With 54.9% A/T in the upstream regions, the chance of **WWW** at any one position is $0.549^4 = 0.091$, so about 8.8 copies are expected in 97 positions, and a random sequence contains at least one with probability 0.994. In information terms, **WWW** carries only 4 bits, but to point at one site among about 100 positions a pattern needs at least $\log_2 100 \approx 6.6$ bits [10]. So the **WWW** search mainly measures how A/T-rich a sequence is, not whether it has a promoter. My data shows this clearly. In 466 of the 1000 genes the upstream region overlaps another gene, and 428 of these come right after another + strand gene, which is how genes are arranged inside an operon (such genes are transcribed from the operon's promoter and usually have no promoter of their own). Still, 99.1% of them came out “positive”. Real σ^{70} promoters are defined by both boxes and the spacer between them, and natural promoters match only 3–4 of the

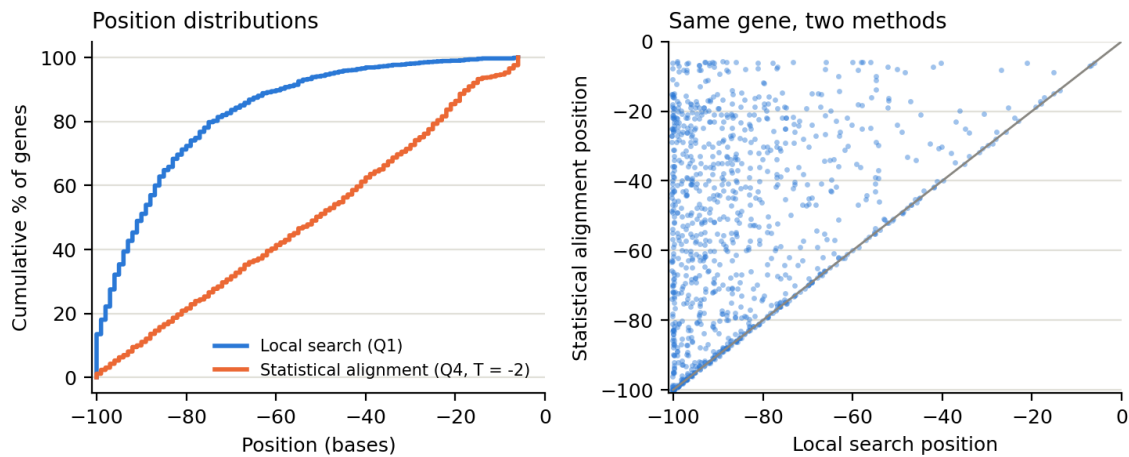


Figure 8: Left: cumulative distribution of the hit position for the two searches. Right: position found by the statistical alignment against the position found by the local search for the same gene (points on the diagonal agree).

6 bases of each consensus [2], so one short degenerate word cannot identify them.

Position distribution. For the same genes, the two searches give very different positions (Figure 8). Only 179 genes (18.0%) have the same position in both and 241 (24.2%) are within 2 bases, and the KS test shows that the two distributions are very different ($D = 0.53$, $p < 10^{-100}$). The local-search positions come from the “first hit” rule: they pile up at -100 and closely follow the shuffled curve. The statistical alignment instead looks at all ~ 12 candidates in a sequence and picks the most PPM-like one. Because a detected window must start with **WWW** and the local search returns the first **WWW**, the statistical hit is always at the same place or closer to the gene (median shift 30 bases), and it sits in a longer A/T run (6.4 against 5.5 bases). Most of its positions (77.6%) are spread evenly between -100 and -26 , like the shuffled sequences (79.1%), but near the gene there is a small difference: more hits between -25 and -14 (16.1% against 12.5% by chance) and between -8 and -6 (4.7% against 3.2%), and fewer between -13 and -9 (1.6% against 5.1%). These regions belong to the translation start region (an A/U-rich stretch before the SD that helps the ribosome to bind [11], the G-rich SD itself and the A-rich spacer before the start codon) rather than to a promoter. If the hits were real -10 boxes I would expect a peak at about -30 to -50 , since the TSS is usually 20–40 bases before the start codon [5]; neither search shows such a peak.

Which search is better? I think the statistical alignment is the better method in principle. It uses every position of the motif, gives partial credit for near matches according to how often each base was observed, ranks all the windows of a sequence, and its threshold lets me trade sensitivity against false positives. In this assignment it could not do better, because its PPM was learned from the output of a non-specific search and holds only 4.06 bits. The thresholds -1 to -5 are also too loose for such a weak matrix: Figure 7 shows that the real and shuffled curves only separate between $T = 0$ and about -0.5 (for example 11.3% against 4.8% at $T = 0$, where only the exact consensus **ATTTTT** passes, and 94.6% against 84.8% at $T = -0.5$). That small gap comes from the extra A/T tracts found in Q2.

Limitations and improvements. If I did this again, I would (i) build the PPM from experimentally confirmed promoters, or at least from the full **TATAAT** and **TTGACA** boxes, instead of from the **WWW** hits; (ii) score windows as log-odds against the genome’s base composition so that A/T-rich DNA does not score high by itself; (iii) search for the -35 and -10 boxes together with a 15–19 base spacer; (iv) set the threshold using shuffled or coding sequences so that the false-positive rate is known; and (v) only use genes that start an operon, because genes inside an operon usually have no promoter.

5 Conclusion

The standard local search found an intact WWW upstream of 99.5% of the first 1000 sense-strand genes of *S. enterica* ASM2286996v1. The hits were mostly far from the genes (median -90) only because the first hit is reported. The A/T runs at the hits were 5.5 bases long on average and longer than expected by chance. The PPM made from the 6 bases at each hit (consensus ATTTT, 4.06 bits) mostly repeated the query, so the statistical alignment detected the same genes (99.0–99.5% for $T = -1$ to -5) but placed the hit somewhere else (median -51 ; same position for only 18% of the genes). Because shuffled sequences give 97.6–97.8%, I conclude that WWW detects A/T-rich DNA rather than real promoters, and that finding promoters reliably needs a more informative model and a threshold that is set against a background.

References

- [1] D. Pribnow, “Nucleotide sequence of an RNA polymerase binding site at an early T7 promoter,” *PNAS*, vol. 72, pp. 784–788, 1975.
- [2] C. B. Harley and R. P. Reynolds, “Analysis of *E. coli* promoter sequences,” *Nucleic Acids Research*, vol. 15, pp. 2343–2361, 1987.
- [3] D. Anastassiou, “Genomic signal processing,” *IEEE Signal Processing Magazine*, vol. 18, no. 4, pp. 8–20, 2001.
- [4] NCBI Datasets, genome assembly ASM2286996v1 (GCA_022869965.1 / GCF_022869965.1). https://www.ncbi.nlm.nih.gov/datasets/genome/GCF_022869965.1/
- [5] A. Mendoza-Vargas *et al.*, “Genome-wide identification of transcription start sites, promoters and transcription factor binding sites in *E. coli*,” *PLoS ONE*, vol. 4, e7526, 2009.
- [6] T. F. Smith and M. S. Waterman, “Identification of common molecular subsequences,” *Journal of Molecular Biology*, vol. 147, pp. 195–197, 1981.
- [7] T. D. Schneider and R. M. Stephens, “Sequence logos: a new way to display consensus sequences,” *Nucleic Acids Research*, vol. 18, pp. 6097–6100, 1990.
- [8] G. D. Stormo, “DNA binding sites: representation and discovery,” *Bioinformatics*, vol. 16, pp. 16–23, 2000.
- [9] J. Shine and L. Dalgarno, “The 3′-terminal sequence of *E. coli* 16S ribosomal RNA: complementarity to nonsense triplets and ribosome binding sites,” *PNAS*, vol. 71, pp. 1342–1346, 1974.
- [10] T. D. Schneider, G. D. Stormo, L. Gold and A. Ehrenfeucht, “Information content of binding sites on nucleotide sequences,” *Journal of Molecular Biology*, vol. 188, pp. 415–431, 1986.
- [11] I. V. Boni, D. M. Isaeva, M. L. Musychenko and N. V. Tzareva, “Ribosome–messenger recognition: mRNA target sites for ribosomal protein S1,” *Nucleic Acids Research*, vol. 19, pp. 155–162, 1991.

A Files in the submitted .zip

- **data/**: the GenBank genome (GCA_022869965.1_ASM2286996v1_genomic.fna) and the protein table (ncbi_dataset.tsv).
- **code/**: `main.py` runs everything in order and `assignment1_notebook.ipynb` runs the same functions step by step. The functions are in `config.py` (settings), `helpers.py`, `step0_data.py`, `step0_methionine_check.py`, `q1_local_search.py`, `q2_consecutive_w.py`, `q3_ppm.py`, `q4_statistical_alignment.py`, `q5_comparison.py`, `plots.py` and `summary.py`.
- **results/**: the derived sequences (`sequences/`, `.fasta` and `.txt`), the results of every question as `.csv` tables (`tables/`), the figures, `summary.json` (all the numbers in this report) and `run_log.txt`. `README.txt` explains how to run the code.

B Main parts of my code

My code is split into one file per step (Appendix A). These are the two functions that do the searches.

Listing 1: Q1: Smith–Waterman local search with the W letter (from `q1_local_search.py`).

```

1 def score(query_letter, base):
2     """+1 if the base fits the query letter (W fits A and T), otherwise -1."""
3     allowed = WEAK if query_letter == "W" else query_letter
4     return MATCH if base in allowed else MISMATCH
5
6
7 def local_search(seq, query):
8     """Smith-Waterman local alignment of the query against seq.
9     Returns (best score, index in seq where the best alignment starts).
10    The query is 'intact' when the best score is len(query) * MATCH, which can
11    only happen when every query letter is matched with no gap."""
12    n, m = len(seq), len(query)
13    H = np.zeros((n + 1, m + 1))
14    for i in range(1, n + 1):
15        for j in range(1, m + 1):
16            H[i, j] = max(0,
17                          H[i - 1, j - 1] + score(query[j - 1], seq[i - 1]), # match / mismatch
18                          H[i - 1, j] + GAP, # gap in the query
19                          H[i, j - 1] + GAP) # gap in the sequence
20
21    best = H.max()
22    # np.argmax gives the FIRST cell with the best score, so when there are many
23    # equal hits I get the one nearest to the 5' end (the same as max() in MATLAB).
24    i, j = np.unravel_index(np.argmax(H), H.shape)
25    while i > 0 and j > 0 and H[i, j] > 0: # trace back to the start of the alignment
26        if H[i, j] == H[i - 1, j - 1] + score(query[j - 1], seq[i - 1]):
27            i, j = i - 1, j - 1
28        elif H[i, j] == H[i - 1, j] + GAP:
29            i -= 1
30        else:
31            j -= 1
32    return best, i

```

Listing 2: Q4: statistical alignment with the PPM (from `q4_statistical_alignment.py`; `log_ppm` is the natural log of the PPM).

```

1 def stat_align(seq, log_ppm):
2     """Score every 6-base window of seq: S = ln p(base1, pos1) + ... + ln p(base6, pos6).
3     I do it for all windows at once with numpy (the same as two for-loops)."""
4     x = to_numbers(seq)
5     n_windows = len(x) - WINDOW + 1
6     scores = np.zeros(n_windows)
7     for j in range(WINDOW):
8         scores += log_ppm[x[j:j + n_windows], j]
9     return scores
10
11
12 # in run_q4(): best window of every sequence and the detection rule
13 best_index, best_rel = [], []
14 for seq in upstream:
15     s = stat_align(seq, log_ppm) # 95 windows, from -100 to -6
16     best_index.append(int(np.argmax(s))) # first best window (5' end side) if there is a tie
17     best_rel.append(s.max() - consensus_score)
18 best_rel = np.array(best_rel)
19 detected = {t: best_rel >= t for t in THRESHOLDS}

```